



[linkedin.com/in/
mohammed-abdul-rasheed](https://www.linkedin.com/in/mohammed-abdul-rasheed)

HANDS-ON AWS PROJECT

Establishing VPC Peering Between Two Public VPCs on AWS

Creating two isolated VPCs with public subnets, establishing a VPC Peering Connection, configuring route tables on both sides, setting up Security Groups, attaching an Elastic IP to Server 1, and verifying cross-VPC connectivity with a live ping test.

2 VPCs	VPC Peering	2 Route Tables	Internet Gateway
2 Security Groups	2 EC2 Instances	Verified via Ping	

Project Overview

The goal of this project was to connect two isolated AWS VPCs using VPC Peering — simulating a real-world scenario where two separate network environments need to communicate securely. **my-peer-public1** (CIDR **10.1.0.0/16**) acts as the requester VPC with an Internet Gateway and an Elastic IP on its EC2 instance. **my-peer-public2** (CIDR **10.2.0.0/16**) acts as the acceptor VPC — it has no Internet Gateway and its EC2 instance has no public IP. Route tables on both VPCs route the peer CIDR through the peering connection. Security Groups enforce access control at the instance level. The design is validated with a live **ping test**.

Component	Name	Configuration
VPC 1	my-peer-public1	10.1.0.0/16
VPC 2	my-peer-public2	10.2.0.0/16
Public Subnet 1	my-peer-publicsubnet1	10.1.1.0/24 (in VPC1)
Public Subnet 2	my-peer-publicsubnet2	10.2.2.0/24 (in VPC2)
Peering Connection	my-peering connection	Active — VPC1 ↔ VPC2
Internet Gateway	Attached to VPC1 only	0.0.0.0/0 → IGW (VPC1 route table)
Route Table 1	my-peer-routes-public1	0.0.0.0/0→IGW 10.2.0.0/16→pcx
Route Table 2	my-peer-routes-public2	10.1.0.0/16→pcx (no IGW)
Security Group 1	my-peer-public1-sg	SSH 22 (0.0.0.0/0) + All Traffic (10.2.2.0/24)
Security Group 2	my-peer-public2-sg	All traffic from VPC1 subnet
EC2 – Server 1	my public server 1	Private: 10.1.1.155 Elastic IP: 52.49.48.249
EC2 – Server 2	my public server 2	Private: 10.2.2.128 No Public IP

1

Architecture Overview

VPC Peering Topology

Two isolated VPCs are connected via a **VPC Peering Connection**. **VPC1** has an Internet Gateway — its EC2 instance has an Elastic IP for external SSH access. **VPC2** has no Internet Gateway — its EC2 instance is reachable only through the peering link from VPC1. Both route tables are updated to route each other's CIDR through the peering connection.

```

INTERNET
|
Internet Gateway (VPC1 only)
|
+-----+
| VPC1: my-peer-public1 (10.1.0.0/16) |
| |
| +-----+ |
| | Public Subnet 1 – 10.1.1.0/24 | | | |
| | Route: 0.0.0.0/0 → IGW | 10.2.0.0/16 → pcx | |
| | |
| | +-----+ | |
| | | my public server 1 (EC2 t2.micro) | | |
| | | Private IP : 10.1.1.155 | | |
| | | Elastic IP : 52.49.48.249 | | |
| | | SG: SSH(22/0.0.0.0/0) + All(10.2.2.0/24) | | |
| | +-----+ | |
| +-----+ |
+-----+
|
VPC Peering Connection (pcx-...)
my-peering connection
Requester: 10.1.0.0/16
Acceptor : 10.2.0.0/16
|
+-----+
| VPC2: my-peer-public2 (10.2.0.0/16) |
| |
| +-----+ |
| | Public Subnet 2 – 10.2.2.0/24 | | | |
| | Route: 10.1.0.0/16 → pcx (NO Internet Gateway) | |
| | |
| | +-----+ | |
| | | my public server 2 (EC2 t2.micro) | | |
| | | Private IP : 10.2.2.128 | | |
| | | No Public IP / No Elastic IP | | |
| | | SG: All Traffic from 10.2.2.0/24 | | |
| | +-----+ | |
| +-----+ |
+-----+

```

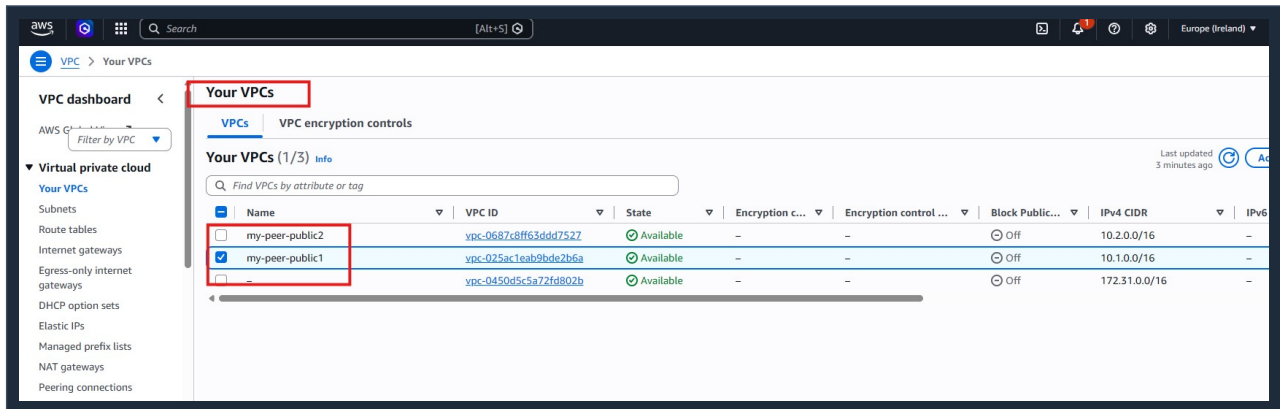
2

Network Foundation

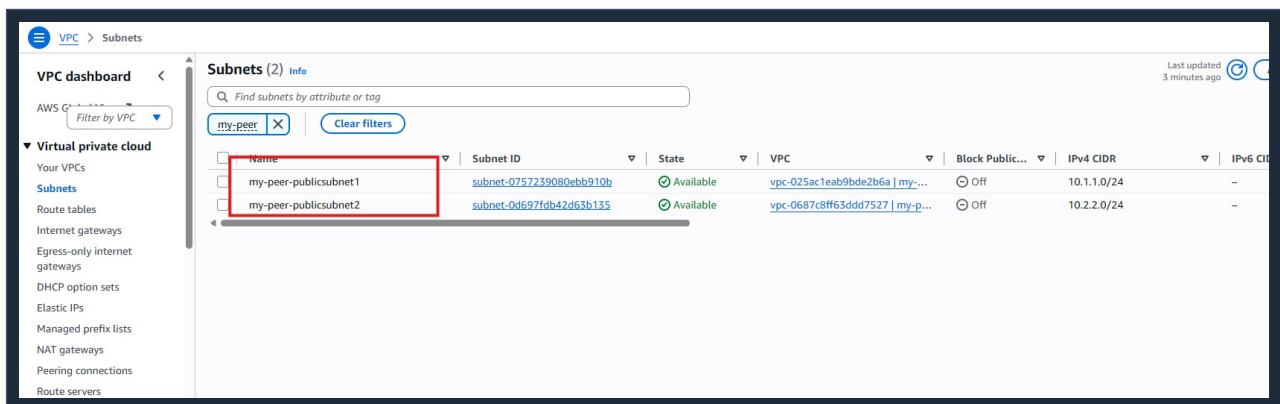
VPCs · Subnets

STEP 01**Create Two VPCs**

Two custom VPCs were created in the **eu-west-1 (Ireland)** region. **my-peer-public1** (CIDR **10.1.0.0/16**) is the requester VPC. **my-peer-public2** (CIDR **10.2.0.0/16**) is the acceptor VPC. Non-overlapping CIDRs are mandatory for VPC Peering.

**STEP 02****Create Public Subnets**

my-peer-publicsubnet1 (10.1.1.0/24) belongs to VPC1. **my-peer-publicsubnet2** (10.2.2.0/24) belongs to VPC2. Both are public subnets — the difference in internet reachability comes from the route tables, not the subnet type.



3

VPC Peering Connection

Requester → Acceptor → Active

STEP 03

Create & Accept Peering Connection

A VPC Peering Connection named **my-peering connection** was initiated from **my-peer-public1** to **my-peer-public2**. Since both VPCs belong to the same AWS account and region, the request was accepted immediately. Status: **Active**. Requester CIDR: 10.1.0.0/16, Acceptor CIDR: 10.2.0.0/16.

■ ■ **Note:** Owner IDs visible in the AWS console have been omitted from this report for security purposes.

The screenshot shows the AWS VPC console interface for peering connections. The left sidebar lists various VPC resources, with 'Peering connections' highlighted. The main panel displays a table of peering connections, showing one connection named 'my-peering connection' with a status of 'Active'. Below the table, the details of the selected connection are shown, including the requester VPC, acceptor VPC, and CIDR blocks. The 'my-peering connection' is highlighted in the left sidebar, and the 'Active' status is shown in a green box. The CIDR blocks 10.1.0.0/16 and 10.2.0.0/16 are also highlighted in a blue box.

Name	Peering connection ID	Status	Requester VPC	Acceptor VPC	Requester CIDRs	Acceptor CIDRs	Requester owner ID
my-peering connection	pcx-0bd9d25ffb57e8cfd	Active	vpc-025ac1eab9bde2b6a / my-...	vpc-0687c8ff63dd7527 / my-...	10.1.0.0/16	10.2.0.0/16	550597787

pcx-0bd9d25ffb57e8cfd / my-peering connection

Details

Requester owner ID: [omitted] Acceptor owner ID: [omitted] VPC Peering connection ARN: [omitted]

my-peering connection

Active

10.1.0.0/16 ↔ 10.2.0.0/16

4

Routing Configuration

Route Tables · Internet Gateway

STEP 04

Update Route Tables for Peering + IGW

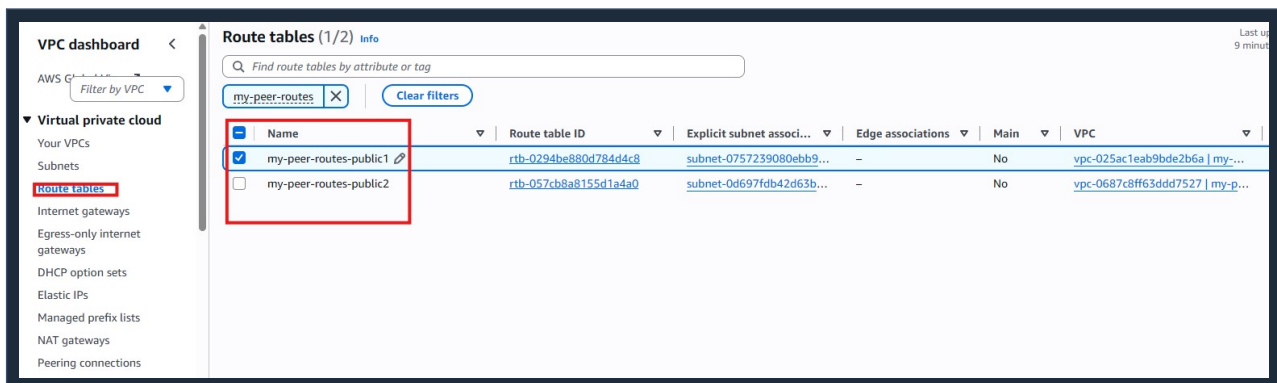
Both route tables were updated to enable cross-VPC communication:

my-peer-routes-public1 (VPC1):

- **0.0.0.0/0** → **Internet Gateway** — gives Server 1 internet access
- **10.2.0.0/16** → **pcx (peering connection)** — routes VPC2 traffic through peering

my-peer-routes-public2 (VPC2):

- **10.1.0.0/16** → **pcx (peering connection)** — routes VPC1 traffic back through peering
- **No Internet Gateway route** — VPC2 remains isolated from the internet



Route Table	Destination	Target	Notes
my-peer-routes-public1	0.0.0.0/0	Internet Gateway	VPC1 internet access
my-peer-routes-public1	10.2.0.0/16	pcx (peering)	VPC1 → VPC2
my-peer-routes-public2	10.1.0.0/16	pcx (peering)	VPC2 → VPC1
my-peer-routes-public2	—	No IGW	VPC2 internet isolated

0.0.0.0/0 → IGW

10.2.0.0/16 → pcx

10.1.0.0/16 → pcx

No IGW for VPC2

5

Security Groups

Instance-level stateful firewall rules

STEP 05

Security Group — my-peer-public1-sg (VPC1)

my-peer-public1-sg is attached to the EC2 instance in VPC1. Inbound rules:

- **SSH (Port 22)** — Source: **0.0.0.0/0** (■ Open to all — for testing only, not production-safe)
- **All Traffic** — Source: **10.2.2.0/24** (VPC2's subnet CIDR) — allows all inbound traffic arriving via the peering connection from Server 2's subnet

The screenshot shows the AWS Management Console for the Security Groups page. The top table lists security groups, with **my-peer-public1-sg** selected. Below, the **Inbound rules (2)** section shows two rules:

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-06ce5041f075c0d02	IPv4	SSH	TCP	22	0.0.0.0/0
-	sgr-002a23919680085c6	IPv4	All traffic	All	All	10.2.2.0/24

SSH Port 22

0.0.0.0/0

All Traffic

10.2.2.0/24

STEP 06

Security Groups List — Both VPCs

Two custom security groups are visible: **my-peer-public1-sg** (VPC1, description: 'allow peering') and **my-peer-public2-sg** (VPC2, description: 'allow'). Each is scoped to its own VPC so traffic is controlled independently.

The screenshot shows the AWS VPC dashboard with the **Security Groups (1/6)** list. The list includes the following security groups:

Name	Security group ID	Security group name	VPC ID	Description
-	sg-0fd44854f034b95a5	default	vpc-0687c8ff63dd47527	default VPC security group
my-peer-public1-sg	sg-0cab87faf87949ca8	my-peer-public1-sg	vpc-025ac1eab9bde2b6a	allow peering
-	sg-0c0862ed91285b60f	default	vpc-025ac1eab9bde2b6a	default VPC security group
-	sg-0ea14831406c7f1cb	default	vpc-0450d5c5a72fd802b	default VPC security group
-	sg-01b8be3fa5b0772fe	www-api-sg	vpc-0450d5c5a72fd802b	launch-wizard-1 created 2026-06-07T0...
my-peer-public2-sg	sg-073b46afc9aec14a	my-peer-public2-sg	vpc-0687c8ff63dd47527	allow

6

Compute & Connectivity

EC2 Instances · Elastic IP

STEP 07

Launch Two EC2 Instances

Two **t2.micro Amazon Linux 2023** instances were launched — one in each VPC's subnet. **my public server 1** is in eu-west-1a (VPC1). **my public server 2** is in eu-west-1b (VPC2) with no public IP. Both show 2/2 status checks passed.

STEP 08

Allocate & Attach Elastic IP to Server 1

Elastic IP **52.49.48.249** was allocated and associated with **my public server 1** (i-0d9ebafca4746d43a). This ensures a persistent public IP regardless of instance restarts. Server 1's private IP remains **10.1.1.155**.

Elastic IP addresses (1) Info							
Find elastic IP addresses by attribute or tag							
☐	Name	Allocated IPv4 address	Type	Allocation ID	Reverse DNS record	Associated instance ID	Private IP address
☐	-	52.49.48.249	Public IP	eipalloc-0ea8b7a890e5edefc	-	i-0d9ebafca4746d43a	10.1.1.155

Elastic IP: 52.49.48.249

Private: 10.1.1.155

Server 2: 10.2.2.128

No Public IP – Server 2

STEP 09

Connect to Server 1 via EC2 Instance Connect

Server 1 was accessed using **EC2 Instance Connect** (browser-based SSH). Connection type: Public IP (52.49.48.249). Username: **ec2-user**. No local key pair needed for this connection method.

EC2 > Instances > i-0d9ebafca4746d43a > Connect to instance

Connect Info

Connect to an instance using the browser-based client.

Instance ID

i-0d9ebafca4746d43a (my public server 1)

VPC ID

vpc-025ac1eab9bde2b6a

Security groups

sg-0cab87faf87949ca8 (my-peer-public1-sg)

IAM role

-

EC2 Instance Connect

SSM Session Manager

SSH client

EC2 serial console

Instance ID

i-0d9ebafca4746d43a (my public server 1)

Connection type

Connect using a Public IP

Connect using a public IPv4 or IPv6 address

Connect using a Private IP

Connect using a private IP address and a VPC endpoint

Public IPv4 address

52.49.48.249

IPv6 address

-

Username

ec2-user

Note: In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Cancel

Connect

7

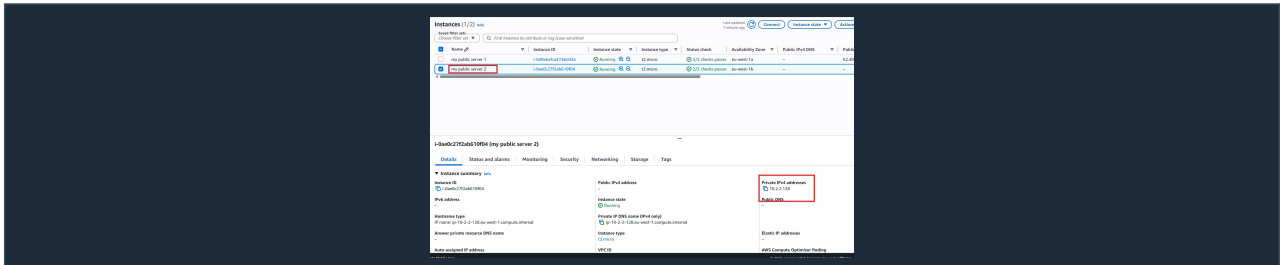
Validation — Ping Test

Cross-VPC Connectivity Confirmed

STEP 10

Server 2 Details — Private IP Only

my public server 2 (i-0ae0c27f2ab610f04) has **Private IPv4: 10.2.2.128** and NO public IPv4 address. It runs in eu-west-1b. This confirms it is completely unreachable from the public internet — accessible only via the peering connection.

**STEP 11**

Ping Test: Server 1 → Server 2 Private IP (10.2.2.128)

From Server 1's terminal, **ping 10.2.2.128** was executed. ICMP replies confirm successful cross-VPC connectivity:

- VPC Peering Connection is routing traffic correctly
- Route tables on both VPCs direct peer traffic through the peering link
- Security Groups allow ICMP between the two instances
- Server 2 responds despite having no public IP or Internet Gateway

```
#_
~\#### Amazon Linux 2023
~~\#####
~~\####|
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~~V~' '->
~~~
~~~.
~~~/_/
~~~/_/

Last login: Sat Jul 4 08:14:57 2026 from 18.202.216.53
[ec2-user@ip-10-1-1-155 ~]$ ping 10.2.2.128
PING 10.2.2.128 (10.2.2.128) 56(84) bytes of data.
64 bytes from 10.2.2.128: icmp_seq=1 ttl=127 time=0.877 ms
64 bytes from 10.2.2.128: icmp_seq=2 ttl=127 time=1.45 ms
64 bytes from 10.2.2.128: icmp_seq=3 ttl=127 time=0.733 ms
64 bytes from 10.2.2.128: icmp_seq=4 ttl=127 time=2.66 ms
64 bytes from 10.2.2.128: icmp_seq=5 ttl=127 time=1.01 ms
64 bytes from 10.2.2.128: icmp_seq=6 ttl=127 time=1.12 ms
□
```

i-0d9ebafca4746d43a (my public server 1)

PublicIPs: 52.49.48.249 PrivateIPs: 10.1.1.155

ping 10.2.2.128

ttl=127

~0.8-2.6 ms

Connectivity Confirmed

Conclusion & Key Takeaways

Successfully established and verified a VPC Peering connection between two AWS VPCs — demonstrating that two completely separate network environments can communicate securely over AWS's internal backbone without traversing the public internet. VPC1 has full internet access via its Internet Gateway while VPC2 remains isolated, yet both servers communicate seamlessly through the peering link.

VPC Peering Basics Allows two VPCs to communicate using private IPs without a VPN or Internet Gateway.	Route Table Updates Both VPCs must update their route tables — pointing the peer CIDR to the pcx target.
One-Sided Internet Gateway Only VPC1 has an IGW. VPC2 stays internet-isolated but is fully reachable from VPC1.	Security Group Control SG1 allows SSH from anywhere (testing) and all traffic from VPC2's CIDR. SG2 allows from VPC1.
Elastic IP for Static Access The Elastic IP on Server 1 gives a permanent public endpoint for SSH testing.	Validated, Not Assumed The design was proven end-to-end with a live ping test — not just configuration checks.

- AWS VPC Peer

VPC
- EC2
- Subnets
- Route Tables

Security Group

Elastic IP
- Internet Gateway